

Chapter 39: Linear Algebra: Essential for Machine Learning and Optimization

Raj Kumar^{1*}

¹Secretary, St. Ignatius School, Aurangabad, Bihar, India

*Corresponding Author Email: kumarjesuit.20@gmail.com

Abstract

This chapter presents a unified overview of the fundamental role of linear algebra in machine learning and optimization. It discusses vectors and matrices, singular value decomposition, spectral theory, optimization landscapes, implicit bias, and convergence of learning algorithms, while connecting these concepts to modern developments such as neural tangent kernels, edge-of-stability, transformers, graph neural networks, matrix completion, and quantum linear algebra. The chapter highlights how eigenvalues, singular values, matrix structure, and spectral properties provide a common mathematical framework for understanding the computation, optimization, and generalization behaviour of contemporary machine-learning models.

Keywords: Linear algebra; Machine learning; Optimization; Spectral theory; Deep learning; Singular value decomposition; Neural networks.

1. Introduction

The field of machine learning has transformed from an engineering discipline to a mathematical science with linear algebra at its heart: from gradient descent to attention mechanisms, almost everything in machine learning comes down to vectors and matrices. Contemporary deep learning phenomena such as edge-of-stability in training, implicit bias towards low-rank solutions, and overparameterization generalization must be explained using spectral properties of loss surfaces and Hessians, something classical convex optimization theory can do little to help with (Boyd & Vandenberghe, 2004).

This chapter unifies all of these ideas under one umbrella narrative, with Section 2 covering the basics, Section 3 covering optimization landscapes' spectral theory, with the recent phenomenon of edge-of-stability included in the discussion, Section 4 covering implicit bias and generalization, Section 5 covering optimization algorithms and how their performance depends on spectral properties of optimization landscape discussed in Section 3, Section 6 covering contemporary deep learning models and other interesting application areas, and finally Section 7 tying everything together.

Note on scope: The goal of this chapter is not to state any novel results. The chapter is merely pedagogical and unifying: the goal is to tie together the results which are well-known by themselves – for example, Jacot et al.'s (2018) Neural Tangent Kernel, Cohen et al.'s (2021) discovery of edge of stability phenomenon, Soudry et al.'s (2018) and Gunasekar et al.'s (2018) work on implicit bias, etc. – into one narrative, so that someone who understands eigendecompositions and Singular Value Decomposition can follow the connection between classical numerical linear algebra and deep learning theory.

Assumed prerequisites are multivariable calculus and linear algebra up to eigendecomposition.

2. Linear Algebra: Computational and Structural Foundations

2.1 Vectorization as Computational Substrate

Data and model parameters in machine learning are represented as vectors and matrices, which enables batched computation across modern hardware (Boyd & Vandenberghe, 2004). For n samples with d features organized into a data matrix $X \in \mathbb{R}^{n \times d}$, predictions for all samples are computed simultaneously as

$$Y = XW + b,$$

replacing n sequential computations with a single matrix operation executed by optimized BLAS libraries near hardware speed (Parr & Howard, 2015). This is not a notational convenience — it is why deep learning is computationally tractable at all.

A matrix $A \in \mathbb{R}^{m \times n}$ has transpose A^T and, for compatible $A \in \mathbb{R}^{m \times k}$, $B \in \mathbb{R}^{k \times n}$, product $C = AB$ with $C_{ij} = \sum_r A_{ir} B_{rj}$. A linear transformation $f(x) = Ax$, $A \in \mathbb{R}^{m \times d}$, together with the L2 norm $\|v\|_2 = \sqrt{\sum_i v_i^2}$ and L1 norm $\|v\|_1 = \sum_i |v_i|$ (the latter inducing sparsity as a regularizer), furnish the vocabulary in which every layer, loss, and regularizer in deep learning is written (Meyer, 2000).

2.2 Matrix Rank, SVD, and the Eckart–Young Theorem

Rank: Rank of a matrix $A \in \mathbb{R}^{m \times n}$ is defined as the dimension of the space spanned by its columns; a rank **one** matrix is an outer product $u^* v^T$, and any matrix of rank k is an outer product of the following form: $A = \sum_{i=1}^k \sigma_i u_i v_i^T$ (Golub & Van Loan, 2013). The low rank nature of data is omnipresent — it occurs in images, recommendation matrices, and time series — with variance concentrated in a few principal directions (Candès & Recht, 2009).

Singular Value Decomposition (SVD): Any $A \in \mathbb{R}^{m \times n}$ may be factorized into

$$A = U \Sigma V^T,$$

with U and V being orthogonal matrices and Σ being a diagonal matrix with its elements satisfying $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ (Golub & Van Loan, 2013). Geometrically, the singular value decomposition of a linear operator consists of a composition of two rotations (V^T and U) with axis-wise stretching (Σ): The unit sphere is transformed into an ellipsoid with the lengths of axes equal to the singular values.

The Eckart–Young theorem: Let $A_k = \sum_{i=1}^k \sigma_i u_i v_i^T$ (rank- k SVD truncation) and B any other rank- k matrix. Then we have $\|A - A_k\|_F \leq \|A - B\|_F$. (Proof omitted due to brevity, for proof see Golub & Van Loan, 2013, Ch. 2, or Eckart & Young, 1936.) This optimality of the SVD is what makes discarding small singular values a principled approach to denoising and compression, not just a heuristic one.

This relates directly to **Principal Component Analysis:** if X are centered data, the covariance $C = \left(\frac{1}{n}\right) X^T X$ can be represented as $X^T X = V \Sigma^2 V^T$, meaning that the right singular vectors of X are also the eigenvectors of its covariance, and $\lambda_k = \sigma_k^2$ (Strang, 2019). PCA, computationally, is an SVD.

Matrix Completion: Given a subset of entries of a low-rank matrix (like in a recommender system), the actual matrix can be recovered from this partial observation exactly by solving the optimization problem $\min \|A\|_* = \sum_i \sigma_i$ (minimization of the nuclear norm, convex relaxation of the rank) subject to matching known values, provided the matrix meets some technical conditions of incoherence

(Candès&Recht, 2009). (Recovery guarantee proof omitted due to brevity; see Candès&Recht, 2009, Thm. 1.3.)

2.3 Linear Systems and Gradient-Based Optimization

The normal equations $X^T X \beta = X^T y$ for the linear regression method are solved using QR factorization of the matrix $A = QR$, where Q is an orthonormal matrix and R is the upper triangular matrix in order to avoid instability issues related to inversion (Trefethen&Bau, 1997). Generally speaking, the minimization of the loss function $L(\theta)$ in the process of training is carried out through the use of gradient descent $\theta \leftarrow \theta - \eta \nabla_{\theta} L$, and for neural networks, the value of $\nabla_{\theta} L$ is calculated through backpropagation, which is an iterative use of the chain rule via matrix–vector multiplications (Parr & Howard, 2015). The process of training a neural network is, mechanically, a series of linear algebraic operations interspersed with nonlinear functions.

3. Spectral Theory and Non-Convex Optimization Landscapes

3.1 The Hessian as a Window into Training Dynamics

The **Hessian** $H = \nabla^2 L$, the matrix of second-order partial derivatives of the loss function, captures the loss function's curvature, which changes significantly during training. At the time of initialization, eigenvalues of the Hessian of a random-weighted neural network are concentrated in a narrow band, which is what one would expect from the Marchenko-Pastur law for random matrices (Marchenko&Pastur, 1967); as training proceeds, some outlier eigenvalues (i.e., isolated eigenvalues) appear and correspond to the directions along which the network learns useful information (Granzio, Zohren, & Roberts, 2022).

Condition number $\kappa = \lambda_{\max}/\lambda_{\min}$ determines convergence rates: small κ corresponds to fast convergence; large κ corresponds to slow and oscillating convergence (Boyd & Vandenberghe, 2004). An empirically well-known observation is that even though deep networks are highly over-parameterized, their condition numbers stay relatively small during training, which allows gradient descent algorithms to work at all in such high-dimensional and non-convex optimization problems (Allen-Zhu, Li, & Song, 2019).

3.2 The Neural Tangent Kernel: Two Training Regimes

For very wide networks, the Neural Tangent Kernel (NTK), $K(x, x') = \langle \nabla_w f(w, x), \nabla_w f(w, x') \rangle$, provides an analyzable linearization of the training dynamics (Jacot, Gabriel, & Hongler, 2018). The eigenstructure of the NTK allows us to distinguish between two training regimes:

- **Lazy training regime (network width $\theta(n)$ relative to n samples):** NTK stays approximately constant throughout training and thus the network acts as a fixed kernel method, and the convergence depends on the smallest eigenvalue of the NTK (Liu, Zhu, & Belkin, 2020).
- **Feature-learning regime (scaling of the network width is superlinear in n):** NTK changes significantly, representations become adapted to the input, and this is the transition that the recently introduced infinite-width parametrizations (like μP) intend to control (Yang & Hu, 2021).

(The exact width threshold for each of the regimes is calculated in Jacot et al., 2018, and Yang & Hu, 2021; calculations not shown for brevity.)

3.3 Loss Landscape Geometry and the Edge of Stability

In high dimensions, non-convex losses are counter-intuitive compared to lower-dimensional intuition: local minima are not strict and most critical points are **saddles**, where the Hessian has both positive and negative eigenvalues and can escape along the negative direction of curvature (Dauphin, Pascanu, Gulcehre, Cho, Ganguli, & Bengio, 2014). In a wide class of over-parameterized problems, all local minima are in fact global minima, with the “no spurious local minima” property proven from the Hessian eigenstructure (Ge, Jin, & Zheng, 2017 proved this for low-rank matrix problems, including matrix sensing and completion; the reasoning applies, with extra assumptions, to over-parameterized neural networks).

There exists a remarkable phase transition in terms of learning rate for the stability of training. The full batch gradient descent usually evolves the maximum eigenvalue of the Hessian until it stays close to the threshold

$$\lambda_{\max} \approx \frac{2}{\eta},$$

which is termed the **edge of stability** (Cohen, Kaur, Li, Kolter, & Talwalkar, 2021). Below this threshold, the training process is stable, with exponentially fast loss decreases. Above it, the loss becomes non-monotonically oscillatory within the short timescale but decreases on the long timescale — the phenomenon which is against classical convex optimization theory and correlates with good generalization performance. There was recently proposed an explanation of this and some other phenomena (such as clustered minima and anomalous diffusion) in the framework of multifractal model of the loss landscape (Ly & Gong, 2025). (The derivation of the dynamics of the edge-of-stability regime is beyond the scope of this chapter; one may refer to Cohen et al., 2021, §3 and Ly & Gong, 2025.)

Table 1 summarizes how each landscape feature traces back to a linear-algebraic root.

TABLE 1: Loss-Landscape Phenomena and Their Spectral Origins

Phenomenon	Spectral Root (Hessian)	Optimization Effect
Local minima	All eigenvalues positive	Rare in high dimension; usually near-global (Ge et al., 2017)
Saddle points	Mixed-sign eigenvalues	Escapable via negative-curvature directions (Dauphin et al., 2014)
Flat regions	Clustered near-zero eigenvalues	Slow convergence; source of implicit bias
Edge of stability	$\lambda_{\max} \approx 2/\eta$	Non-monotonic loss, improved generalization (Cohen et al., 2021)

4. Implicit Bias, Feature Learning, and Generalization

A major mystery in deep learning is how it generalizes when models have sufficient capacity to interpolate data, going against classical learning theory intuition (Bartlett, Long, Lugosi, & Tsigler, 2020). The explanation lies in the notion of **implicit bias**: gradient descent does not select any random interpolating solution, but a particular one chosen by its optimization dynamics.

For linear models, gradient descent from zero initialization converges to the minimum-norm solution among all those that fit the data (Boyd & Vandenberghe, 2004; Soudry, Hoffer, Nacson, Gunasekar, & Srebro, 2018, showed the same result for logistic loss with maximum-margin interpretation on separable data). For linear convolutional neural networks, however, gradient descent from zero initialization seeks to minimize a frequency-based norm, which is an entirely different inductive bias (Gunasekar, Lee, Soudry, & Srebro, 2018). In both cases, the underlying principle is spectral: learned weight matrices show fast singular-value decay, $\sigma_i \propto i^{-\alpha}$, even with huge numbers of parameters, limiting model complexity.

Mathematically, this is equivalent to solving an implicit optimization problem:

$$\min \|W\|_* \text{ Subject to fitting the training data,}$$

Where $\|\cdot\|_*$ is the nuclear norm, the convex relaxation of the rank used in matrix completion (Section 2.2). Generalization bounds have an inherent spectral structure: bounds based on the margin $\gamma = 1/\|w\|_2$ and on the effective rank of feature matrices become tighter with increasing spectral decay rate (Bartlett et al., 2020).

The summary: ***over-parameterized models do not interpolate randomly — implicit spectral bias of gradient descent chooses smooth, low-rank solutions out of a solution space that is hugely underdetermined***, explaining such diverse phenomena as matrix completion, maximum margin for classification and practical success of hugely over-parameterized deep learning models.

5. Optimization Algorithms: First-Order, Second-Order, and Randomized Methods

5.1 First-Order Methods and Spectral Convergence

Parameter updates are performed via $w_{t+1} = w_t - \eta \nabla L(w_t)$, where η is the learning rate (Boyd & Vandenberghe, 2004). The convergence speed is determined by the Hessian spectrum, as $O\left(\exp\left(-\frac{t}{\kappa}\right)\right)$ in the convex case, with $\kappa = \lambda_{\max}/\lambda_{\min}$ (Nesterov, 2018). High condition numbers entail an unfortunate trade-off: the learning rate should be low to ensure stability for high-curvature directions, thus taking extremely small steps in low-curvature directions. Empirically, over-parameterized networks manage to keep moderate condition numbers during training – which is one reason gradient descent remains a sensible method in such settings (Allen-Zhu, Li, & Song, 2019).

Momentum techniques consist of accumulating gradients, $v_t = \beta v_t - 1 + \nabla L(w_t)$, followed by the update of $w_t - \eta v_t$, which serves as a low-pass filter damping oscillations along high-curvature directions. Nesterov acceleration reaches the optimal convex rate of $O(1/\sqrt{\kappa})$ (Nesterov, 2018). The Adam method adjusts the learning rate per coordinate based on gradient variance (Kingma & Ba, 2015) – an approximation of a diagonal preconditioner ignoring off-diagonal Hessian correlations.

The stability of the update is ensured by $\eta \lambda_{\max} < 2$ in the convex case; in non-convex training this same criterion governs the edge-of-stability transition discussed in Section 3.3 (Cohen, Kaur, Li, Kolter, & Talwalkar, 2021). This provides us with a natural heuristic for learning rates: estimating $\lambda_{\max}$ through power iteration and using $\eta \approx 2/(3\lambda_{\max})$, placing us just within the stable regime. Spectral learning rate diagnostics for adaptive optimizers remain an active area of research (Cohen, Ghorbani, Krishnan, Agarwal, & Gilmer, 2022).

5.2 Second-Order Methods and Curvature

Newton's method takes the full Hessian into account: $w_{t+1} = w_t - H^{-1} \nabla L(w_t)$, converging quadratically, i.e., $O(\log \log 1/\epsilon)$ iterations for sufficiently conditioned problems (Nesterov, 2018). However, the computation and inversion of H require $O(n^3)$ complexity, making the approach prohibitively expensive after n goes above millions of dimensions (Boyd & Vandenberghe, 2004). Quasi-Newton algorithms (BFGS, L-BFGS) approximate H^{-1} only using gradient information, thus lowering complexity to $O(n^2)$ or $O(nk)$ for some small constant k (Nocedal & Wright, 2006). They are not used much in deep learning due to the non-convexity problem and violation of the update assumptions based on exact gradient information.

Natural gradient descent scales the update by the inverse Fisher information matrix, $F = E[\nabla \log p(y|x) \nabla^T \log p(y|x)]$ (Amari, 1998), which corresponds to gradient descent under a Riemannian manifold equipped with the Fisher metric; K-FAC algorithm estimates F^{-1} through Kronecker factorization (Martens & Grosse, 2015). In practice, one should opt for a second-order optimization procedure when the solution has an adequately conditioned Hessian or when computational accuracy is prioritized over memory capacity. First-order methods are preferred when the latter is limited, the true solution is of global rank comparable to that of first-order methods (Section 4), or the noise in gradients is high (Nocedal & Wright, 2006).

5.3 Randomized and Distributed Optimization

While random projections sacrifice exactness for speed and efficiency, they guarantee to preserve geometry in a provable manner: Johnson-Lindenstrauss lemma proves that projection of vectors from $\mathbb{R}^d$ to $\mathbb{R}^k$, where $k = O(\log n/\epsilon^2)$, preserves distances up to factor of $(1 \pm \epsilon)$ with high probability (Martinson & Tropp, 2020), and, importantly, preserves eigenvalues and eigenvectors of the object being sketched (Woodruff, 2014). This property of sketching is the one that allows sketched least squares problem (minimizing $\|SAx - Sb\|$ where S is $k \times m$ sketch $S, k \ll m$) to inherit the condition number, and therefore convergence properties, of the original problem. **Leverage-score sampling** provides another step: rows of $A = U\Sigma V^T$ are chosen according to $\ell_i = \|U_i\|^2$ (rows' alignments with dominant singular directions), leading to $(1 + \epsilon)$ -approximation using $O(n)$ samples rather than $O(n^2)$ of uniform sampling (Drineas, Magdon-Ismail, Mahoney, & Woodruff, 2012). Finally, stochastic gradient descent could be seen in a similar light: each mini-batch is implicitly random sketch of the covariance matrix of gradients, and large mini-batches provide better spectral approximation at the cost of additional computations.

The idea behind distributed optimization is averaging of gradient steps computed by each agent with the help of their network neighbors. Convergence of such process depends on the second smallest eigenvalue $\lambda_2(L)$ of graph Laplacian of communication network and is of order $O\left(\frac{\kappa}{\lambda_2} \cdot T\right)$ (Boyd et al., 2011; Nedić & Ozdaglar, 2009). Complete graph has maximal value of λ_2 , but requires n communication links per node; path graph requires minimum communication effort, but has $\lambda_2 = O\left(\frac{1}{n^2}\right)$; sparse expander graph requires only $O(n)$ links per node and has constant $\lambda_2 = \Theta(1)$. Finally, gradient quantization in federated learning is essentially adding noise to this spectral decomposition of communication graph (Konečný et al., 2016).

6. Modern Architectures and Advanced Topics

6.1 Transformers as Linear Algebra Machines

The transformer layer is composed of learned linear mappings: input projects to query, key, and value matrices; attention is computed as $\text{softmax}(QK^T/\sqrt{d_k})V$, which is a stochastic (row-normalized) matrix that corresponds to the learned attention over context positions; followed by a feed-forward network of two dense layers. The singular-value analysis of attention matrices reveals that different heads learn complementary patterns, whereas the others are largely redundant — in practice, many heads can be removed with little loss of accuracy (Michel, Levy, &Neubig, 2019). This explains the interest in low-rank approximation of attention that allows reducing the computation time while retaining the expressive capacity of the model (Wang, Li, Khabsa, Fang, & Ma, 2020).

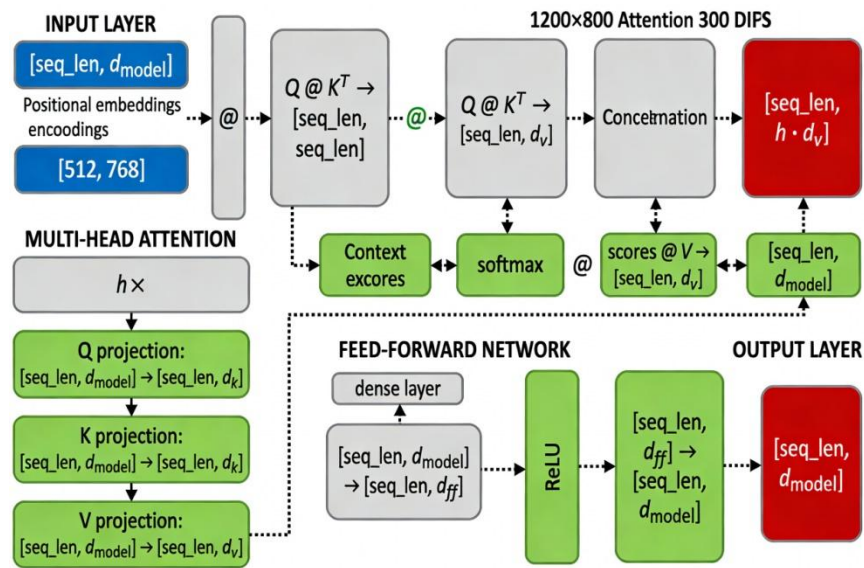


Figure 1: Transformer Architecture — Matrix Operations and Dimension Flow

Vision Transformers operate similarly to transformer layers but applied to the image patches, hence having the complexity $O(n^2)$ with respect to the number of patches. Graph Neural Networks perform the spectral filtering of the graph Laplacian; expressivity depends on the spectral gap of the graph. The problem with poorly connected graphs is the “over-squashing” — the compression of information from distant nodes into the fixed-size vector is too fast for propagation (Alon&Yahav, 2021).

Architectural choice is a trade-off between complexity/expressiveness depending on attention mechanism. Dense attention is $O(n^2)$, capturing the global structure, kernelized linear attention is $O(n)$ with reduced expressivity, and sparse/local attention is $O(n \log n)$, trading off the global context for efficiency. Table 2 below gives an overview of different choices.

TABLE 2: Architecture Complexity Comparison

Architecture	Complexity	Key Linear-Algebra Operation	Best For
Standard Transformer	$O(n^2)$	Dense QK^T attention	Medium sequences, global context
Linear Transformer	$O(n)$	Kernelized / low-rank attention	Very long sequences
Graph Neural Network	$O(edges)$	Spectral filtering / message passing	Graph-structured data

6.2 Matrix Completion and Quantum Linear Algebra

The recommendation system problem can be seen as that of recovering the whole matrix from a few randomly sampled entries; if the true matrix has a rank of r , it has only $O(r(m+n))$ degrees of freedom as opposed to mn , and the number of observations needed for exact recovery becomes as small as $O(rn \log n)$ through nuclear norm minimization, i.e., solving $\min_X \{\|X\|_* / (X = \text{observed entries})\}$ (Candès & Recht, 2009), which corresponds to the same convex relaxation of rank described in Section 2.2. Another approach is to perform non-convex decomposition of the form $X \approx UV^T$, which does not involve any relaxation but uses gradient descent instead, whose implicit bias toward low-norm solutions tends to converge to the global minimum, an example of benign non-convexity (Gunasekar, Woodworth, Bhojanapalli, Neyshabur, & Srebro, 2017). Robust PCA is a similar approach to handle noise by performing the decomposition $M = L + S$ into low-rank L and sparse outlier S components, used for instance for video background subtraction (Candès, Li, Ma, & Wright, 2011).

Quantum computing provides a totally different way to the same underlying linear algebra. The HHL algorithm solves $Ax = b$ in $O(\log n)$ time for sparse, well-conditioned Hermitian matrix A (Harrow, Hassidim, & Lloyd, 2009); Quantum Singular Value Transformation (Gilyén, Su, Low, & Wiebe, 2019) generalizes it further to a wider class of matrix functions. However, in practice the overhead due to input-output bottlenecks – mapping of the classical data onto quantum states and vice versa – often destroys apparent exponential speedup for general problems, and true quantum advantage is limited to matrices with particular structure (e.g., sparse, low-rank, or block-diagonal); classical algorithms currently dominate for the general problem of deep learning. This explains the shift of research focus from quantum machine learning to these more structured problems (Tang, 2022).

7. Conclusion and Synthesis

The key idea, articulated over the last six sections, is that linear algebra is not peripheral to machine learning, but essential to it: Data are vectors and matrices (Section 2); Optimization dynamics are driven by the eigenvalues of the Hessian (Section 3); The generalization of over-parametrized models occurs due to spectral bias towards low-rank, smooth solutions without explicit regularization (Section 4); And each of the algorithms and architectures that enable modern deep learning – momentum and adaptive methods, distributed optimization, transformers, graph neural networks, matrix completion, and even quantum linear-algebra algorithms – is just another way of taking advantage of that very same spectral structure (Sections 5-6).

Three main ideas run throughout the chapter. **First, Convergence:** Regardless of whether we use gradient descent, Newton's method, or randomized sketching, the rate at which an algorithm converges to its solution is determined by the condition number of the matrix involved (Nesterov, 2018). **Second, Generalization:** Over-parameterized models don't memorize random patterns because gradient descent biases its solutions towards those which have small norm within the manifold of solutions that perfectly fit the training data (Soudry et al., 2018; Gunasekar et al., 2018). **Third, The edge of stability:** Training doesn't just converge smoothly downhill, but organizes itself around a particular sharpness threshold defined by the learning rate – a decidedly non-classical phenomenon whose explanation lies in the spectral analysis, not classical convex theory (Cohen et al., 2021; Ly & Gong, 2025).

But this theory is incomplete. Finite-width feature learning in multi-layer networks, the landscape of realistic (non-toy) loss functions, and when exactly quantum linear-algebra algorithms give actual rather than asymptotically real computational advantages (Tang, 2022) are among the problems unsolved yet. This chapter is not about answering these questions but about giving a unified perspective of a single phenomenon that underlies all of them and other known results presented above.

References

1. Allen-Zhu, Z., Li, Y., & Song, Z. (2019). A convergence theory for deep learning via over-parameterization. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, PMLR 97, 242–252.
2. Alon, U., & Yahav, E. (2021). On the bottleneck of graph neural networks and its practical implications. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
3. Amari, S. (1998). Natural gradient works efficiently in learning. *Neural Computation*, 10(2), 251–276.
4. Bartlett, P. L., Long, P. M., Lugosi, G., & Tsigler, A. (2020). Benign overfitting in linear regression. *Proceedings of the National Academy of Sciences*, 117(48), 30063–30070.
5. Boyd, S., & Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press.
6. Boyd, S., Parikh, N., Chu, E., Peleato, B., & Eckstein, J. (2011). Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1), 1–122.
7. Candès, E. J., & Recht, B. (2009). Exact matrix completion via convex optimization. *Foundations of Computational Mathematics*, 9, 717–772.
8. Candès, E. J., Li, X., Ma, Y., & Wright, J. (2011). Robust principal component analysis? *Journal of the ACM*, 58(3), 1–37.
9. Cohen, J. M., Kaur, S., Li, Y., Kolter, J. Z., & Talwalkar, A. (2021). Gradient descent on neural networks typically occurs at the edge of stability. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
10. Cohen, J. M., Ghorbani, B., Krishnan, S., Agarwal, N., Medapati, S., Badura, M., Suo, D., Cardoze, D., Nado, Z., Dahl, G. E., & Gilmer, J. (2022). Adaptive gradient methods at the edge of stability. *arXiv preprint arXiv:2207.14484*.
11. Dauphin, Y. N., Pascanu, R., Gulcehre, C., Cho, K., Ganguli, S., & Bengio, Y. (2014). Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. *Advances in Neural Information Processing Systems*, 27, 2933–2941.
12. Drineas, P., Magdon-Ismael, M., Mahoney, M. W., & Woodruff, D. P. (2012). Fast approximation of matrix coherence and statistical leverage. *Journal of Machine Learning Research*, 13, 3475–3506.
13. Eckart, C., & Young, G. (1936). The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3), 211–218.
14. Ge, R., Jin, C., & Zheng, Y. (2017). No spurious local minima in nonconvex low rank problems: A unified geometric analysis. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, PMLR 70, 1233–1242.

15. Gilyén, A., Su, Y., Low, G. H., & Wiebe, N. (2019). Quantum singular value transformation and beyond: Exponential improvements for quantum matrix arithmetics. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing (STOC)*.
16. Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations* (4th ed.). Johns Hopkins University Press.
17. Granzio, D., Zohren, S., & Roberts, S. J. (2022). A random matrix theory approach to neural network learning rates. *Journal of Machine Learning Research*, 23(81), 1–61.
18. Gunasekar, S., Lee, J. D., Soudry, D., & Srebro, N. (2018). Implicit bias of gradient descent on linear convolutional networks. *Advances in Neural Information Processing Systems*, 31.
19. Gunasekar, S., Woodworth, B., Bhojanapalli, S., Neyshabur, B., & Srebro, N. (2017). Implicit regularization in matrix factorization. *Advances in Neural Information Processing Systems*, 30.
20. Harrow, A. W., Hassidim, A., & Lloyd, S. (2009). Quantum algorithm for linear systems of equations. *Physical Review Letters*, 103(15), 150502.
21. Jacot, A., Gabriel, F., & Hongler, C. (2018). Neural tangent kernel: Convergence and generalization in neural networks. *Advances in Neural Information Processing Systems*, 31.
22. Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
23. Konečný, J., McMahan, H. B., Yu, F. X., Richtárik, P., Suresh, A. T., & Bacon, D. (2016). Federated learning: Strategies for improving communication efficiency. *arXiv preprint arXiv:1610.05492*.
24. Liu, C., Zhu, L., & Belkin, M. (2020). On the linearity of large non-linear models: When and why the tangent kernel is constant. *Advances in Neural Information Processing Systems*, 33.
25. Ly, A., & Gong, P. (2025). Optimization on multifractal loss landscapes explains a diverse range of geometrical and dynamical properties of deep learning. *Nature Communications*, 16(1), 3252.
26. Marchenko, V. A., & Pastur, L. A. (1967). Distribution of eigenvalues for some sets of random matrices. *Matematicheskii Sbornik*, 114(4), 507–536.
27. Martens, J., & Grosse, R. (2015). Optimizing neural networks with Kronecker-factored approximate curvature. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, PMLR 37, 2408–2417.
28. Martinsson, P.-G., & Tropp, J. A. (2020). Randomized numerical linear algebra: Foundations and algorithms. *Acta Numerica*, 29, 403–572.
29. Meyer, C. D. (2000). *Matrix Analysis and Applied Linear Algebra*. SIAM.
30. Michel, P., Levy, O., & Neubig, G. (2019). Are sixteen heads really better than one? *Advances in Neural Information Processing Systems*, 32.
31. Nedić, A., & Ozdaglar, A. (2009). Distributed subgradient methods for multi-agent optimization. *IEEE Transactions on Automatic Control*, 54(1), 48–61.
32. Nesterov, Y. (2018). *Lectures on Convex Optimization* (2nd ed.). Springer.
33. Nocedal, J., & Wright, S. J. (2006). *Numerical Optimization* (2nd ed.). Springer.
34. Parr, T., & Howard, J. (2015). The matrix calculus you need for deep learning. *arXiv preprint arXiv:1802.01528*.
35. Soudry, D., Hoffer, E., Nacson, M. S., Gunasekar, S., & Srebro, N. (2018). The implicit bias of gradient descent on separable data. *Journal of Machine Learning Research*, 19(70), 1–57.
36. Strang, G. (2019). *Introduction to Linear Algebra* (6th ed.). Wellesley-Cambridge Press.

37. Tang, E. (2022). Quantum-inspired classical algorithms for machine learning. *Communications of the ACM*, 65(11), 68–74.
38. Trefethen, L. N., & Bau, D., III. (1997). *Numerical Linear Algebra*. SIAM.
39. Wang, S., Li, B. Z., Khabsa, M., Fang, H., & Ma, H. (2020). Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*.
40. Woodruff, D. P. (2014). Sketching as a tool for numerical linear algebra. *Foundations and Trends in Theoretical Computer Science*, 10(1–2), 1–157.
41. Yang, G., & Hu, E. J. (2021). Feature learning in infinite-width neural networks. *arXiv preprint arXiv:2011.14522*.